A Hybrid Image Analysis Pipeline for Automatic Slide Extraction from Lecture Video

AutoSlides Technical Report

April 26, 2026

Abstract

This report describes a hybrid image analysis design for extracting presentation slides from lecture video recordings. The problem is formulated as a sequence selection and post-processing task over video frames: the system must identify stable slide states, remove redundant or irrelevant captures, classify ambiguous visual content, and optionally localize the slide canvas for cropping. The proposed design combines structural similarity analysis, perceptual hashing, prompt-guided semantic classification, and geometric slide-canvas detection. Structural Similarity Index Measure (SSIM) is used for online change detection and temporal stability verification. A DCT-based perceptual hash is used for duplicate removal and exclusion-list matching. Semantic filtering separates true slide frames from non-slide frames and recoverable presentation-editing views through either language-model prompting or a local classifier. Finally, a two-stage crop detector combines classical edge geometry with a learned object detector fallback. Mathematical definitions, decision rules, and pseudocode are provided for each stage.

Keywords: lecture video analysis, slide extraction, SSIM, perceptual hash, DCT, semantic filtering, prompt engineering, Canny edge detection, object detection, automatic cropping

1 Introduction

Lecture videos often contain long segments in which the screen region is visually stable, interrupted by slide changes, cursor movement, animations, video playback, application switches, and occasional non-presentation content. Automatically recovering a clean set of slides from such videos is therefore not simply a frame sampling problem. A robust pipeline must distinguish meaningful slide transitions from transient visual noise, remove duplicates, filter non-slide frames, and preserve recoverable images such as presentation software edit-mode screenshots.

The design presented here follows a layered strategy. Low-level temporal analysis is used first because it is fast, deterministic, and suitable for online processing. Perceptual hashing is then used after extraction to remove near-duplicates and known undesirable frames. Semantic classification is reserved for cases where pixel-level similarity is insufficient. Finally, geometric crop detection is applied when a valid slide appears inside a larger application window or screen recording context.

This document intentionally avoids implementation-specific code. Instead, it describes the mathematical model, algorithmic decisions, and pseudocode needed to reproduce the image analysis design in an independent system.

2 Problem Formulation

Let a lecture video be represented as a time-indexed sequence of RGB frames:

$$V = \{I_t \mid t \in \mathcal{T}\}, \qquad I_t \in [0, 255]^{H \times W \times 3}.$$

The objective is to produce an ordered set of extracted slide images:

$$\mathcal{S} = \{S_1, S_2, \ldots, S_n\},$$

where each S_i corresponds to a visually meaningful and temporally stable slide state.

The pipeline has four major objectives:

1. **Temporal detection:** identify frame times at which the visible slide content changes.
2. **Redundancy reduction:** remove repeated or near-identical captures.
3. **Semantic filtering:** distinguish valid slide frames from non-slide or recoverable edit-mode frames.
4. **Spatial normalization:** localize and crop the slide canvas when the slide is embedded in a larger screen.

The complete analysis can be viewed as a composition:

$$\mathcal{S}_{final} = C\left(F_{\text{sem}}\left(F_{\text{hash}}\left(E_{\text{ssim}}(V)\right)\right)\right),$$

where E_{ssim} extracts candidates, F_{hash} removes duplicates and exclusions, F_{sem} performs semantic filtering, and C optionally applies crop normalization.

3 Temporal Slide Detection with SSIM

3.1 Frame Preprocessing

For efficiency and noise reduction, frames are first downsampled:

$$\tilde{I}_t = R(I_t; w_d, h_d),$$

where R is an image resizing operator and (w_d, h_d) is a fixed analysis resolution.

The resized RGB image is converted to grayscale luminance:

$$Y_t(x, y) = 0.299 R_t(x, y) + 0.587 G_t(x, y) + 0.114 B_t(x, y).$$

This luminance channel is used for structural comparison because slide changes are generally expressed as changes in layout, text, and diagram structure rather than chromatic variation alone.

3.2 Structural Similarity

Given two grayscale images X and Y with N pixels, define:

$$\mu_X = \frac{1}{N}\sum_{i=1}^{N} X_i, \qquad \mu_Y = \frac{1}{N}\sum_{i=1}^{N} Y_i,$$

$$\sigma_X^2 = \frac{1}{N}\sum_{i=1}^{N}(X_i - \mu_X)^2, \qquad \sigma_Y^2 = \frac{1}{N}\sum_{i=1}^{N}(Y_i - \mu_Y)^2,$$

$$\sigma_{XY} = \frac{1}{N}\sum_{i=1}^{N}(X_i - \mu_X)(Y_i - \mu_Y).$$

The Structural Similarity Index Measure is:

$$\mathrm{SSIM}(X, Y) = \frac{(2\mu_X\mu_Y + C_1)(2\sigma_{XY} + C_2)}{(\mu_X^2 + \mu_Y^2 + C_1)(\sigma_X^2 + \sigma_Y^2 + C_2)}.$$

For 8-bit image intensity values, stability constants are selected as:

$$C_1 = (0.01 \cdot 255)^2, \qquad C_2 = (0.03 \cdot 255)^2.$$

A candidate slide change is detected when the current frame is sufficiently dissimilar from the last accepted slide:

$$\mathrm{SSIM}(Y_t, Y_\ell) < \tau,$$

where Y_ℓ is the luminance image of the most recently saved slide and τ is the similarity threshold.

3.3 Adaptive Thresholding

The threshold τ controls sensitivity. Higher values detect smaller changes but are more likely to react to noise; lower values are more tolerant but may miss subtle slide edits. A practical adaptive threshold can be expressed as:

$$\tau(c) = \begin{cases} \tau_{\text{loose}}, & c \in \mathcal{C}_{lowquality}, \\ \tau_{\text{normal}}, & \text{otherwise}, \end{cases}$$

where c denotes contextual metadata such as classroom or capture environment. Typical values are:

$$\tau_{\text{strict}} = 0.999, \qquad \tau_{\text{normal}} = 0.9987, \qquad \tau_{\text{loose}} = 0.998.$$

3.4 Temporal Stability Verification

A single changed frame may correspond to an animation, cursor movement, transient video frame, or application switch. To reduce false positives, the pipeline verifies that a candidate frame remains stable across subsequent checks.

Let C be the candidate frame detected at time t_c, and let $F_1, \ldots, F_k$ be the next k sampled frames. The candidate is accepted only if:

$$\mathrm{SSIM}(C, F_j) \geq \tau \quad \forall j \in \{1, \ldots, k\}.$$

If any verification frame differs significantly from C, the candidate is rejected and the system waits for a new stable state.

3.5 Sampling Interval Under Playback Speed

Let Δ_0 be the configured detection interval at normal playback speed and r be playback rate. The wall-clock sampling interval is:

$$\Delta(r) = \max\left(\Delta_{\min}, \frac{\Delta_0}{g(r)}\right),$$

where $g(r)$ is a bounded scaling function. For moderate speeds, $g(r) = r$ preserves an approximately constant video-time interval. For very high speeds, $g(r)$ is capped:

$$g(r) = \min(r, r_{\max}).$$

This prevents timer instability while keeping high-speed processing practical.

3.6 Pseudocode

```
Algorithm 1: Online slide candidate extraction

Input:
  video frames I_t
  SSIM threshold tau
  verification count k
  sampling interval Delta

Output:
  ordered candidate set S

S <- empty list
last <- null
state <- idle
candidate <- null
verified <- 0

for each sampled frame I_t do
    Y_t <- preprocess(I_t)

    if last is null then
        save I_t into S
        last <- Y_t
        continue
    end if

    if state = verifying then
        if SSIM(candidate, Y_t) >= tau then
            verified <- verified + 1
            if verified >= k then
                save candidate source frame into S
                last <- candidate
                state <- idle
            end if
```

```
        else
            state <- idle
        end if
        continue
    end if

    if SSIM(last, Y_t) < tau then
        candidate <- Y_t
        verified <- 0
        state <- verifying
    end if
end for
```

4 Perceptual Hashing for Post-Processing

4.1 DCT-Based Hash Construction

After candidate extraction, perceptual hashing is used to compare images by low-frequency visual structure. Let I be a candidate slide image. The image is converted to grayscale and resized to $N \times N$, for example $N = 64$. Its two-dimensional Discrete Cosine Transform (DCT) is:

$$D_{u,v} = \frac{2}{N}\alpha(u)\alpha(v)\sum_{x=0}^{N-1}\sum_{y=0}^{N-1} I_{x,y}\cos\left(\frac{(2x+1)u\pi}{2N}\right)\cos\left(\frac{(2y+1)v\pi}{2N}\right),$$

where:

$$\alpha(0) = \frac{1}{\sqrt{2}}, \qquad \alpha(k) = 1 \text{ for } k > 0.$$

The low-frequency block of size $m \times m$ is retained, for example $m = 16$. The DC coefficient is omitted, leaving a vector of AC coefficients:

$$\mathbf{d} = [D_1, D_2, \ldots, D_{m^2-1}].$$

Let:

$$\eta = \text{median}(\mathbf{d}).$$

The perceptual hash is the binary vector:

$$h_i = \begin{cases} 1, & D_i \geq \eta, \\ 0, & D_i < \eta. \end{cases}$$

For $m = 16$, this yields a 255-bit vector if the DC coefficient is omitted. In practice the serialized representation may be padded to a fixed storage width.

4.2 Hamming Distance

Given two perceptual hashes $h^{(a)}$ and $h^{(b)}$, similarity is measured by Hamming distance:

$$d_H(h^{(a)}, h^{(b)}) = \sum_i h_i^{(a)} \oplus h_i^{(b)}.$$

Two images are treated as near-duplicates when:

$$d_H(h^{(a)}, h^{(b)}) \leq \delta,$$

where δ is a small integer threshold. This is particularly useful for repeated slide states, temporarily revisited pages, and stable classroom error screens.

4.3 Duplicate Removal and Exclusion Matching

The same distance rule supports two related tasks:

1. **Duplicate removal:** compare each new hash with previously retained hashes and remove it if a near-duplicate already exists.
2. **Exclusion matching:** compare each hash with a curated set $\mathcal{E}$ of known undesirable images, such as black screens, no-signal frames, or desktop backgrounds.

The exclusion rule is:

$$\exists e \in \mathcal{E} : d_H(h, e) \leq \delta \quad \Longrightarrow \quad \text{remove image.}$$

```
Algorithm 2: pHash post-processing

Input:
  extracted images S
  exclusion hashes E
  Hamming threshold delta

Output:
  retained images R
  removed images T

R <- empty list
T <- empty list
seen <- empty list

for each image s in S do
    h <- perceptual_hash(s)

    if exists e in E such that Hamming(h, e) <= delta then
        move s to T with reason "exclusion"
        continue
    end if

    if exists q in seen such that Hamming(h, q.hash) <= delta then
        move s to T with reason "duplicate"
        continue
    end if

    append s to R
    append (s, h) to seen
end for
```

5 Semantic Filtering

5.1 Motivation

SSIM and pHash are intentionally low-level. They do not know whether a frame is a real full-screen slide, a browser page, a desktop background, a media player, or a presentation-editing interface. Semantic filtering addresses this limitation by assigning each remaining image to a label set:

$$\mathcal{L} = \{\texttt{slide}, \texttt{not_slide}, \texttt{may_be_slide_edit}\}.$$

The third class is important because edit-mode screenshots may still contain valid slide content that can be recovered through cropping.

5.2 Prompt-Guided LLM Classification

A multimodal language model can be used as a semantic classifier by converting the classification problem into a constrained visual question-answering task. The prompt is not merely an instruction string; it acts as a formal interface contract between the image analysis pipeline and the language model. A useful prompt can be decomposed as:

$$P = R \oplus C^+ \oplus A^+ \oplus C^- \oplus C^e \oplus O,$$

where R is the role instruction, C^+ defines positive slide criteria, A^+ lists acceptable presentation artifacts, C^- defines non-slide categories, C^e defines edit-mode presentation views, and O specifies a machine-parseable output schema.

The positive slide criteria are intentionally narrow:

$$C^+ = \{\text{full-screen presentation}, \text{title/bullets/figures/charts}, \text{professional slide layout}\}.$$

However, the artifact set A^+ prevents over-filtering. A slide should remain positive even if it contains letterboxing, a visible cursor, small navigation controls, pen tools, a timer, or a slide-number overlay. This distinction is important because lecture recordings often preserve presentation-mode controls that are not part of the slide content but do not invalidate the frame.

The negative class C^- is broad and enumerative. It includes office applications outside presentation mode, code editors, terminals, browsers, file explorers, video players, meeting interfaces, whiteboard applications with extensive tool palettes, blank screens, loading pages, error messages, webcam views, and mixed multi-window desktop states. Enumerating common false positives helps the model place ambiguous visual interfaces outside the slide class.

The edit-mode criterion C^e is separated from general non-slide content. It covers authoring applications such as PowerPoint, Keynote, or Google Slides when visible user-interface chrome is present:

$$\begin{aligned} C^e = \{&\text{ribbon/toolbars}, \text{slide thumbnail side panel}, \\ &\text{property inspector}, \text{notes pane}, \text{title/menu/window chrome}\}. \end{aligned}$$

Images satisfying C^e are not full-screen slides, but they may contain a recoverable slide canvas. Therefore, in the three-class policy they are mapped to `may_be_slide_edit` rather than `not_slide`.

Two Prompt Variants

The prompt family supports two label spaces. The simple variant uses:

$$\mathcal{L}_2 = \{\texttt{slide}, \texttt{not_slide}\},$$

and folds edit-mode presentation views into `not_slide`. The distinguish variant uses:

$$\mathcal{L}_3 = \{\texttt{slide}, \texttt{not_slide}, \texttt{may_be_slide_edit}\},$$

and preserves edit-mode frames as recoverable removals.

The variant selection can be written as a projection:

$$\pi_{\text{simple}}(y) = \begin{cases} \texttt{not_slide}, & y = \texttt{may_be_slide_edit}, \\ y, & \text{otherwise}, \end{cases}$$

while the distinguish mode uses the identity mapping:

$$\pi_{\text{distinguish}}(y) = y.$$

Single-Image and Batch Schemas

The output contract O is deliberately strict. For a single image, the model is asked to return exactly one JSON object:

```
{"classification": "slide"}
```

or, in the distinguish variant:

```
{"classification": "may_be_slide_edit"}
```

For a batch of n images, the expected output is an indexed map:

$$O_n = \{\texttt{image_0} : y_0, \texttt{image_1} : y_1, \ldots, \texttt{image_n-1} : y_{n-1}\},$$

where:

$$y_i \in \mathcal{L}_2 \quad \text{or} \quad y_i \in \mathcal{L}_3.$$

The explicit zero-based indexing aligns each output label with the corresponding input image and avoids natural-language explanations that would make downstream parsing fragile.

```
Algorithm 3: Prompt-guided semantic classification

Input:
  image batch B = [I_0, ..., I_{n-1}]
  mode in {simple, distinguish}

Output:
  labels y_0, ..., y_{n-1}

if mode = simple then
    L <- {slide, not_slide}
    prompt <- role + slide criteria + artifacts
              + non-slide criteria + JSON schema over L
else
    L <- {slide, not_slide, may_be_slide_edit}
    prompt <- role + slide criteria + artifacts
              + edit-mode criteria + non-slide criteria
```

```
              + JSON schema over L
end if

response <- multimodal_LLM(prompt, B)
labels <- parse_json(response)

for each label y_i do
    if y_i not in L then
        mark image I_i as undecided
    end if
end for
```

5.3 Local Classifier Output

A learned classifier produces logits:

$$\mathbf{z} = [z_1, z_2, z_3],$$

which are converted to class probabilities by softmax:

$$p_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}.$$

The predicted class and confidence are:

$$\hat{y} = \arg\max_i p_i, \qquad \gamma = \max_i p_i.$$

5.4 Conservative Decision Policy for Local ML

Because false removals are more harmful than false retentions, the decision policy is conservative. Let θ_L be a low trust threshold, θ_H be a high trust threshold, and θ_S be a slide-probability safeguard.

If $\hat{y} = \texttt{slide}$, the image is retained. If $\hat{y} \neq \texttt{slide}$, the decision is:

$$D(\hat{y}, \gamma, p_{\texttt{slide}}) = \begin{cases} \texttt{slide}, & \gamma < \theta_L, \\ \hat{y}, & \gamma > \theta_H, \\ \hat{y}, & \theta_L \leq \gamma \leq \theta_H \text{ and } p_{\texttt{slide}} < \theta_S, \\ \texttt{slide}, & \text{otherwise.} \end{cases}$$

This rule keeps low-confidence non-slide predictions and only removes ambiguous images when the probability of being a slide is also low.

Parameter	Typical value	Purpose
θ_L	0.75	keep low-confidence non-slide predictions
θ_H	0.90	trust high-confidence non-slide predictions
θ_S	0.25	retain ambiguous cases with sufficient slide probability

Table 1: Thresholds for conservative semantic filtering.

5.5 Two-Class and Three-Class Decision Modes

Some deployments may prefer a two-class policy:

$$\mathcal{L}_2 = \{\texttt{slide}, \texttt{not_slide}\}.$$

In this mode, edit-mode detections are folded into `not_slide`. In the three-class policy:

$$\mathcal{L}_3 = \{\texttt{slide}, \texttt{not_slide}, \texttt{may_be_slide_edit}\},$$

edit-mode images are preserved as recoverable removals. This distinction improves user review workflows because recoverable frames can be routed to automatic cropping instead of being treated as ordinary non-slide images.

6 Slide-Canvas Localization and Automatic Cropping

6.1 Problem Definition

Given an image I that may contain a slide canvas embedded inside a larger screen, the crop task is to estimate an axis-aligned bounding box:

$$B = (x, y, w, h)$$

such that the rectangle tightly contains the slide content while excluding application chrome, side panels, toolbars, and black borders.

6.2 Black Border Stripping

Screen recordings often contain letterboxing. Let G be a grayscale image. A top border row r is considered black if:

$$\frac{1}{W}\sum_{x=1}^{W} G(r, x) < \beta,$$

where β is a blackness threshold. The same rule applies to bottom rows and left/right columns. Scanning is limited to a fraction ρ of image width or height to avoid removing meaningful dark content.

6.3 Canny-Geometry Detector

After border stripping, edges are extracted using Canny edge detection. At a high level, the Canny response can be interpreted as:

$$E = \mathrm{Canny}(G; t_L, t_H),$$

where t_L and t_H are low and high hysteresis thresholds. A small dilation operation connects fragmented slide borders.

Contours are then extracted from E. A contour candidate q is accepted only if it satisfies geometric constraints:

$$\mathrm{vertices}(q) = 4,$$
$$a_{\min} \le \frac{w_q h_q}{WH} \le a_{\max},$$
$$\frac{\mathrm{area}(q)}{w_q h_q} \ge \phi_{\min}.$$

The first condition prefers rectangular boundaries, the second excludes tiny or excessively large boxes, and the third favors contours that substantially fill their bounding rectangles.

6.4 Aspect Ratio Scoring

Most slides are close to either $16:9$ or $4:3$. Let:

$$\mathcal{A} = \left\{ \frac{16}{9}, \frac{4}{3} \right\}.$$

For a candidate box with aspect ratio $r = w/h$, define:

$$e(r) = \min_{a \in \mathcal{A}} \frac{|r - a|}{a}.$$

The aspect score is:

$$s_{\text{aspect}}(r) = \max\left(0, 1 - \frac{e(r)}{\epsilon}\right),$$

where ϵ is an aspect tolerance. The final candidate score is:

$$s(B) = s_{\text{aspect}}(w/h) \cdot \frac{wh}{WH}.$$

The selected crop is:

$$B^* = \arg\max_{B \in \mathcal{B}} s(B).$$

6.5 Learned Detector Fallback

Classical edge geometry is precise when a clean slide border is visible, but it can fail under occlusion, low contrast, or complex authoring interfaces. A learned detector can be used as a fallback. The detector predicts boxes in center format:

$$(\hat{c}_x, \hat{c}_y, \hat{w}, \hat{h}, \hat{p}),$$

where $\hat{p}$ is confidence. The box is converted to corner format:

$$x_1 = \hat{c}_x - \frac{\hat{w}}{2}, \qquad y_1 = \hat{c}_y - \frac{\hat{h}}{2},$$

$$x_2 = \hat{c}_x + \frac{\hat{w}}{2}, \qquad y_2 = \hat{c}_y + \frac{\hat{h}}{2}.$$

Predictions below a confidence threshold κ are discarded. Remaining boxes are filtered by non-maximum suppression (NMS). For boxes A and B:

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

If $\text{IoU}(A, B) > \lambda$, the lower-confidence box is suppressed.

```
Algorithm 4: Hybrid automatic crop detection

Input:
  image I
  classical detector parameters
  learned detector confidence kappa
  NMS threshold lambda
```

```
Output:
  crop box B or null

G <- grayscale(I)
G_inner <- strip_black_borders(G)
E <- canny_edges(G_inner)
C <- contours(E)
B_classical <- best_rectangular_candidate(C)

if B_classical is not null then
    return map_to_original_coordinates(B_classical)
end if

P <- learned_detector(I)
P <- remove boxes with confidence < kappa
P <- non_maximum_suppression(P, lambda)

if P is empty then
    return null
else
    return highest_confidence_box(P)
end if
```

7 Integrated Pipeline

The complete workflow combines the previous stages:

1. Sample video frames according to playback speed and detection interval.
2. Use SSIM to detect structural changes.
3. Verify temporal stability before accepting a new candidate.
4. Run perceptual hashing to remove duplicates and match exclusions.
5. Apply semantic filtering to classify slide, non-slide, and edit-mode frames.
6. Optionally crop recoverable edit-mode or manually selected images.
7. Preserve removed and cropped images with reversible metadata.

The ordering is deliberate. Cheap deterministic methods run first, reducing the number of images sent to expensive semantic or geometric modules. User-recoverable categories are preserved rather than deleted, which makes the system robust to classification uncertainty.

8 Evaluation Considerations

A practical evaluation should separate the four objectives:

- **Change detection recall:** fraction of true slide transitions captured.

- **Duplicate precision:** fraction of removed duplicates that are actually redundant.
- **Semantic filtering safety:** false removal rate for true slides.
- **Crop quality:** intersection-over-union between predicted crop boxes and manually annotated slide canvases.

For slide extraction, false negatives are often more costly than false positives because a missed slide cannot be recovered from the final set without reprocessing the video. Therefore, thresholds should be tuned to favor recall in early stages and precision in removal stages. The use of a recoverable trash manifest further reduces the cost of conservative or uncertain decisions.

9 Conclusion

The proposed image analysis design treats automatic slide extraction as a hybrid temporal, perceptual, semantic, and geometric problem. SSIM provides fast online transition detection. DCT-based perceptual hashing removes redundancy and known visual artifacts. Semantic classification separates true slides from non-slide content while preserving recoverable edit-mode frames. Hybrid crop detection combines the precision of classical edge geometry with the robustness of learned detection. Together, these stages form a practical pipeline for converting noisy lecture video into a clean, reviewable, and exportable slide collection.

References

[1] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.

[2] N. Ahmed, T. Natarajan, and K. R. Rao, "Discrete cosine transform," *IEEE Transactions on Computers*, vol. C-23, no. 1, pp. 90–93, 1974.

[3] J. Canny, "A computational approach to edge detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-8, no. 6, pp. 679–698, 1986.

[4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 779–788, 2016.